

Planning in AI

PDDL

Example STRIPS

Axiomatize the following scenario, appropriately with action precondition and effect axioms, and obtain successor state axioms.

Fluents:

- **boxClosed(s)** - The box is closed in situation s.
- **robotCloseToBox(s)** - The robot is close to the box in situation s.
- **objectInsideBox(s)** - The object is inside the box in situation s.

*Task: extract the object from the box
i.e.*

Goal: not objectInsideBox

Actions:

- **moveCloseToBox** - The robot moves close to the box. This can be done if the robot is not close to the box, and has the effect that the robot will be close to the box.
- **openBox** - The robot opens the box. This can be done if the robot is close to the box and the box is closed, and has the effect that the box will be open (that is, not closed).
- **extractObject** - The robot extracts the object inside the box. This requires that the box is open and the robot is close to the box, and has the effect that the object will be outside the box.

Initial situation description: Initially the robot is not close to the box, the box is closed, and the object is inside the box.



Example STRIPS

domain

```
(define (domain box)
  (:requirements :strips :negative-preconditions)
  (:predicates (boxClosed)
               (robotCloseToBox)
               (objInsideBox))
)

(:action moveCloseToBox
  :parameters ()
  :precondition (and (not (robotCloseToBox)))
  :effect (and (robotCloseToBox))
)

(:action openBox
  :parameters ()
  :precondition (and (robotCloseToBox) (boxClosed))
  :effect (and (not (boxClosed)))
)

(:action extractObject
  :parameters ()
  :precondition (and (not (boxClosed)) (robotCloseToBox))
  :effect (and (not (objInsideBox)))
))
```

problem

```
(define (problem box-1)
  (:domain box)
  (:objects )

  (:init
    ;; (not (robotCloseToBox))
    (boxClosed)
    (objInsideBox)
  )

  (:goal
    (not (objInsideBox))
  ))
```

Example ADL (Adapted from Situation Calculus)

We consider 3 action each with its preconditions and effects (sort of postconditions):

- $drive(t, \ell_1, \ell_2)$
 - ▶ **Pre:** $at(t, \ell_1, s) \wedge connected(\ell_1, \ell_2)$
 - ▶ **Effects:** $at(t, \ell_2, s') \wedge \neg at(t, \ell_1, s')$ where $s' = do(drive(t, \ell_1, \ell_2), s)$
- $load(p, t)$
 - ▶ **Pre:** $\exists \ell. at(p, \ell, s) \wedge at(t, \ell, s) \wedge \neg \exists t'. on(p, t', s)$
 - ▶ **Effects:** $\forall \ell. \neg at(p, \ell, s') \wedge on(p, t, s')$ where $s' = do(load(p, t), s)$
- $unload(t)$
 - ▶ **Pre:** $\exists p. on(p, t, s)$
 - ▶ **Effects:** $(\forall p, \ell. on(p, t, s) \wedge at(t, \ell, s) \rightarrow at(p, \ell, s')) \wedge$
 $(\forall p. on(p, t, s) \rightarrow \neg on(p, t, s'))$
NB: equivalent to $\forall p. on(p, t, s) \rightarrow ((\forall \ell. at(t, \ell, s) \rightarrow at(p, \ell, s')) \wedge \neg on(p, t, s'))$
 where $s' = do(unload(t), s)$

Example

```
(define (domain logistics)
```

```
(:requirements :adl)
```

```
(:types
```

```
  pack vehicle - object
```

```
  truck - vehicle
```

```
  loc
```

```
)
```

```
(:predicates
```

```
  (connected ?l1 ?l2 - loc) ;; nonfluent
```

```
  (at ?p - object ?l - loc) ;; fluent
```

```
  (on ?p - pack ?t - truck) ;; fluent
```

```
)
```

domain

```
(:action load
```

```
  :parameters (?p - pack ?t - truck)
```

```
  :precondition (exists (?l - loc) (and (at ?t ?l) (at ?p ?l)))
```

```
  :effect (and
```

```
    (on ?p ?t)
```

```
    (forall (?l - loc) (not (at ?p ?l))))
```

```
)
```

```
)
```

```
(:action drive
```

```
  :parameters (?t - truck ?from ?to - loc)
```

```
  :precondition (and (at ?t ?from) (connected ?from ?to))
```

```
  :effect (and (at ?t ?to) (not (at ?t ?from))))
```

```
)
```

```
(:action unload
```

```
  :parameters (?t - truck)
```

```
  :precondition (exists (?p - pack) (on ?p ?t))
```

```
  :effect (and
```

```
    (forall(?l - loc ?p - pack) (when (and (on ?p ?t) (at ?t ?l)) (at ?p ?l))))
```

```
    (forall(?p - pack) (when (on ?p ?t) (not (on ?p ?t)))))
```

```
)
```

```
)
```

Example



```
(define (problem logistics-1)
  (:domain logistics)
  (:objects
    truck1 - truck
    package1 - pack
    package2 - pack
    package3 - pack
    location1 - loc
    location2 - loc
    location3 - loc
    location4 - loc
  )
```

```
(:init
  (at truck1 location1)
  (at package1 location1)
  (at package2 location1)
  (at package3 location1)

  ;; no atoms involving on

  (connected location1 location2)
  (connected location2 location3)
  (connected location3 location4)
  (connected location4 location1)
  (connected location1 location4)
  (connected location2 location4)
)
```

```
(:goal (and
  (at package1 location4)
  (at package2 location4)
  (at package3 location4))
)
```

problem

Planning in AI

PLANNING IN NONDETERMINISTIC DOMAINS

Example

Consider the following variant of the Yale Shooting Scenario, which has to do with a shooting a turkey named Fred, with a gun that can be defective. Find a plan to kill the turkey.

Yale Shooting Scenario variant:

- **Fluents:**
 - **alive** - The turkey is alive in the current situation.
 - **loaded**– The gun is loaded in the current situation.
 - **jammed**– The gun is jammed in the current situation.
- **Actions:**
 - **load** – Load the gun. Requires that the gun is not loaded; has two alternative possible effects:
 - The gun is loaded and is not jammed
 - The gun is loaded and is jammed
 - **shoot** - Shoot the gun. Requires that the gun be loaded; has a conditional effect:
 - If the gun is not jammed, then it unloads the gun and kills the turkey (not alive)
 - If the gun is jammed, then it unjams it
 - **wait** – A no-op; Can always be performed and has no effect on any fluent
- **Initial situation description:**
 - Initially the turkey is alive, the gun is not loaded, and it is not jammed.
- **Goal:**
 - Kill the turkey, i.e., turkey is not alive

Use PDDL “oneof” for expressing alternative effects.

Example

- **Fluents** (propositional):
alive, loaded, jammed

- **Actions:**

load

Pre: not loaded

Effects:

oneof

loaded and not jammed

loaded and jammed

shoot

Pre: loaded

Effects:

if not jammed *then* not loaded and not alive

if jammed *then* not jammed

wait

Pre: none

Effects: none

- **Initial situation description:**

alive and not loaded and not jammed

- **Goal:**

not alive

Example

```
(define (domain yale-shooting-nondeterministic)
  (:requirements :adl :non-deterministic)
```

```
  (:predicates
    (alive) ;; fluent
    (loaded) ;; fluent
    (jammed) ;; fluent
  )
```

```
(define (problem yale-1)
  (:domain box yale-shooting-nondeterministic)
  (:objects )
```

```
  (:init
    (alive)
    ;;(not loaded))
    ;;(not jammed))
  )
```

```
  (:goal
    (not (alive))
  ))
```

problem

```
(:action load
  :parameters ()
  :precondition (not (loaded))
  :effect (oneof
    (and (loaded) (jammed))
    (and (loaded) (not (jammed)))
  )
)
```

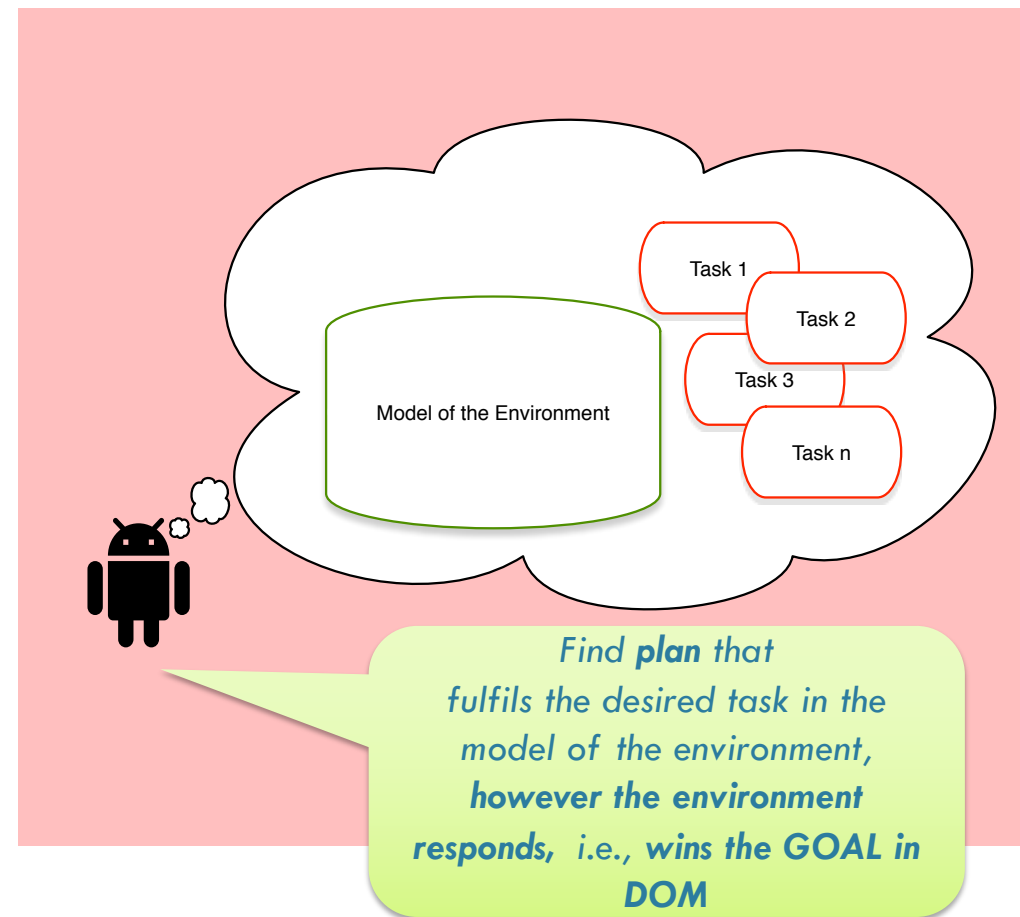
domain

```
(:action shoot
  :parameters ()
  :precondition (loaded)
  :effect (and
    (when (not (jammed)) (and (not loaded) (not alive)))
    (when (jammed) (not (jammed)))
  )
)
```

```
(:action wait
  :parameters ()
  :precondition ()
  :effect ()
)
```

Planning in Nondeterministic Domains

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - PDDL
 - **Domain** is (or better generates) a **non-deterministic transition system** (i.e., a **game arena for two players agent and env!**)
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan**/behavior/policy/strategy that fulfills **GOAL** in **DOM**



Planning in Nondeterministic Domains

Nondeterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

*It's a **game arena** between agent controlling actions and env controlling fluents in the domain!*

Who controls what?

Fluents controlled by **environment**

Actions controlled by **agent**

Observe: $\delta(s, a, s')$

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = **game** between two players:

agent tries to force eventually reaching G no matter how other **environment** behave.

Plan = **strategy** to **win** the game.

*Fully Observable Nondeterministic Domains
Strong Planning, i.e., considers env adversarial*

(FOND_{sp} is EXPTIME-complete)

Planning in Nondeterministic Domains

Winning states for nondeterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(adversarial preimage of a set $\mathcal{E}$)** the following function:

$$PreAdv(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \forall s' \in \mathcal{S}. \delta(s, a, s') \supset s' \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreAdv(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreAdv(W)
}
return W
  
```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \forall s'. \delta(s, a, s') \supset s' \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in AI

PLANNING IN NONDETERMINISTIC DOMAINS WITH STOCHASTIC FAIRNESS

Example

Consider the following another variant of the Yale Shooting Scenario, which has to do with a shooting a turkey named Fred, with a gun that can be defective. Find a plan to kill the turkey.

Yale Shooting Scenario variant:

- **Fluents:**
 - **alive** - The turkey is alive in the current situation.
 - **loaded** – The gun is loaded in the current situation.
- **Actions:**
 - **load** – Load the gun. Requires that the gun is not loaded; has two alternative possible effects:
 - The gun is loaded
 - nothing
 - **shoot** - Shoot the gun. Requires that the gun be loaded; has the following effect:
 - it unload the gun and kills the turkey (not alive)
 - **wait** – A no-op; Can always be performed and has no effect on any fluent
- **Initial situation description:**
 - Initially the turkey is alive, and the gun is not loaded.
- **Goal:**
 - Kill the turkey, i.e., turkey is not alive

*Use PDDL “**oneof**” for expressing alternative effects.*